

How to create a product requirements document (PRD)



> By Dan Radigan, Atlassian Head of Technical Product Marketing

Get started with the free product requirements template

Refine product requirements, validate use cases, and guide your team through key pre-launch and post-launch checks.

[Use template](#)

Key Takeaways

- A product requirements document (PRD) defines the purpose, features, and behavior of a product, aligning stakeholders and guiding development.
- Agile PRDs focus on shared understanding, customer needs, and flexibility, avoiding overly detailed specs.
- Effective PRDs include goals, assumptions, user stories, design, and clear out-of-scope items, fostering collaboration and adaptability.
- Collaborate with your team to create and regularly update a concise PRD, ensuring clarity and alignment throughout the project.

Building a successful product starts with clear direction and alignment. That's where a product requirements document (PRD) comes in.

A PRD outlines a product's essential features, goals, and functionality, which serves as a [product roadmap](#) for teams to turn ideas into reality. This is a crucial step for any [product manager](#) needing to align teams, stakeholders, and the overall goals.

In this article, we'll break down what a product requirements document is, why it matters, and how it sets the foundation for effective product development.

What is a product requirements document (PRD)?

A product requirements document (PRD) defines the product to be build—outlining the product's purpose, features, functionalities, and behavior.

It defines the product's purpose, key [product features](#), user needs, and success criteria, providing a single source of truth for cross-functional teams.

By clearly documenting requirements and expectations, a PRD helps ensure everyone—from designers and developers to stakeholders—stays aligned throughout the product development process.



Product requirements document for Agile work

What does the requirements gathering process look like in an agile world? It sounds tricky—and it is. But don't worry.

At [Atlassian](#), we know all about creating PRDs in an agile environment. Agile requirements are a product owner's best friend.

Product owners who don't use agile requirements get caught up with spec'ing out every detail to deliver the right software (then cross their fingers hoping they've spec'ed out the right things). On the other hand, Agile requirements also depend on a shared understanding of the customer.

This is shared between the product owner, designer, and the development team. That shared understanding and empathy for the target customer unlocks hidden bandwidth for product owners.

They can focus on higher-level requirements and leave implementation details to the development team, who is fully equipped to do so because of the shared understanding.

Tips for creating an effective Product requirements document

If you're excited by the idea of a shared understanding, but haven't a clue how to create it, try some of these tips:

- When conducting customer interviews, include a member of the design and development teams so they can hear from a customer directly instead of relying on the product owner's notes. It will also give them the chance to probe deeper while the topic is fresh in the customer's mind.
- Make developing and using customer personas a team effort. Each team member has unique perspectives and insights, and needs to understand how the personas influences product development.
- Make work item triage and [backlog grooming](#) a team sport as well. These are great opportunities to make sure everyone is on the same page, and understand why the product owner has prioritized work the way they have.

Want to give it a try? [Sign up for Confluence](#).

Create a [customer interview template](#) to document your customer insights. Follow our tutorial to get started on conducting valuable customer interviews:

- Create insightful [customer interview](#) pages
- Turn info into insights with the Customer Interview Pyramid



ANTI-PATTERNS TO WATCH FOR

- The entire project is already spec'd out in great detail before any engineering work begins
- Thorough review and iron-clad sign-off from all teams are required before work even starts
- Designers and developers don't know when requirements have been updated
- Requirements are never updated in the first place (because everyone signed off on them, remember?)
- The product owner writes requirements without the participation of the team

You've discussed a set of user stories with your engineer and designer. Gone back and forth, had a few whiteboard sessions, and concluded there are a few more dimensions you need to consider for this feature that you are working on.

You need to flesh out some assumptions you're making, think deeper about how this fits in the overall scheme of things and keep track of all the open questions you need to answer. What next?

What should a PRD include?

When writing a requirements document, it's helpful to use a consistent template across the team so everyone can follow along and give feedback. At Atlassian, we use [Confluence](#) to create product requirements with the [product requirements document template](#).

We've found that the section below provides "just enough" context to understand a project's requirements and its impact on users:

1. Define project specifics

Project plan

Driver	@mention the driver
Approver	@ approver
Contributors	@ contributors
Informed	@ stakeholders
Objective	Summarize the objective in 1-2 sentence
Due date	Type // to add a date
Key outcomes	List expected outcomes and success metrics
Status	



We recommend including high-level direction at the top of the page as follows:

- Participants: Who is involved? Include the product owner, team, stakeholders
- Status: What's the current state of the program? On target, at risk, delayed, deferred, etc.
- Target release: When is it projected to ship?

2. Team goals and business objectives

Goals		
# Filter by tag Status Owner Team Following Reporting line		
435 Goals		
Goal	Status	Due date
▼ 🎯 1 million cards issued in the first 6 months	ON TRACK 0.7	📅 August
🎯 Grow to 3500 monthly issued cards	ON TRACK 0.7	📅 October
🎯 Banc.ly referral program - user access	OFF TRACK 0.3	📅 Jul-Sep
▼ 🎯 10 million unique site visits	AT RISK 0.5	📅 July
🎯 Grow engaged users >80% MoM	AT RISK 0.4	📅 August
🎯 Banc.ly Web: Increasing API adoption	PENDING	📅 May

Get straight to the point. Inform, but don't bore. Having the right software to detail these goals—in an easy to read view—is helpful for all involved.

Business objectives need to be clear and to the point, but also informative enough to align stakeholders. Don't give opportunities for others to make assumptions.

3. Background and strategic fit

The background section explains the motivation behind the product or feature and how it aligns with broader company goals. It provides context for why the project is important and what problems it aims to solve.

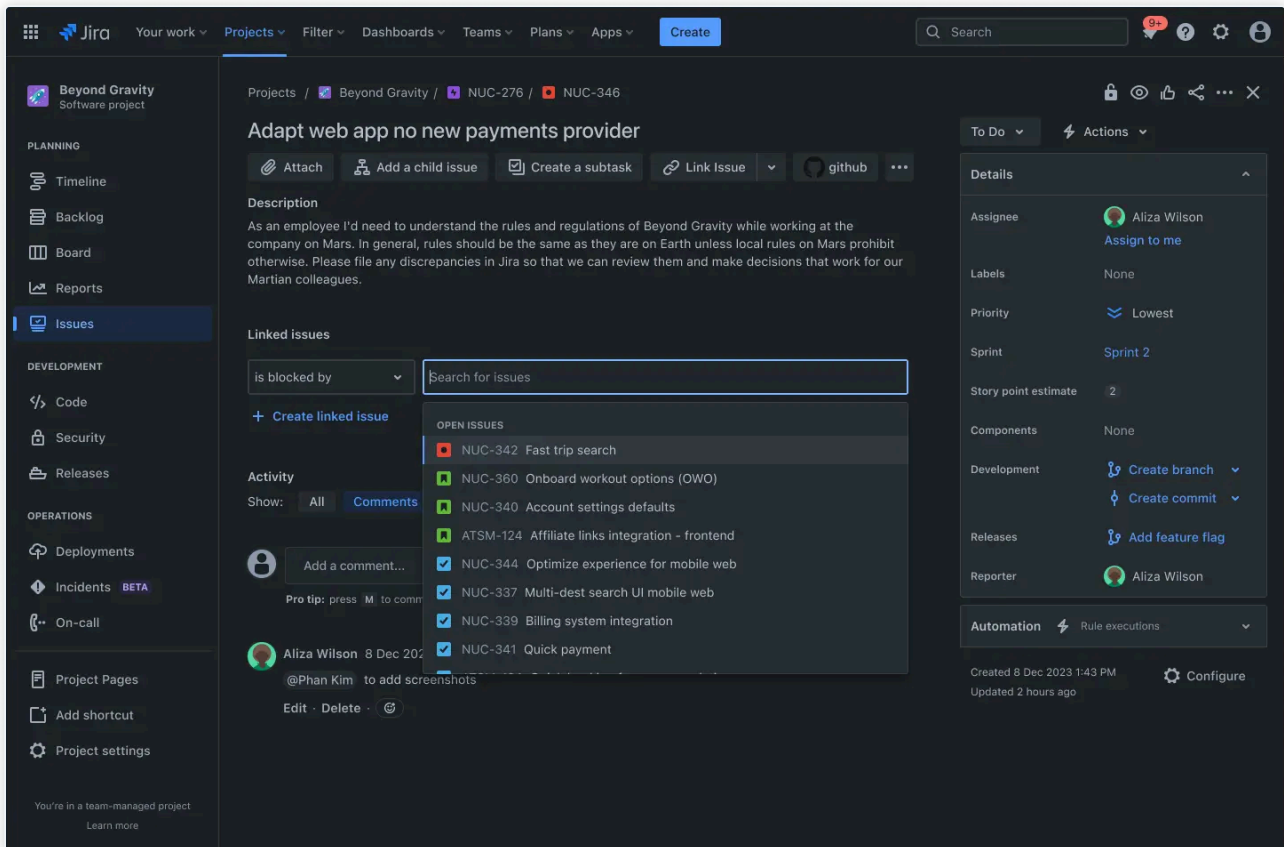
Detailing the strategic fit ensures that everyone understands how this work supports the organization's vision and priorities. This clarity helps teams stay focused on delivering value that matters.

4. Assumptions

This section outlines any assumptions the team is making about technology, business needs, or user behavior. Clearly stating assumptions helps identify potential risks and areas that may need validation.

It also ensures that everyone is aware of the factors influencing decisions and planning. Revisiting these assumptions throughout the project can help teams adapt as new information emerges.

5. User Stories



List or link to the [user stories](#) involved. Also link to customer interviews, and included screenshots of what you've seen. Provide enough detail to make a complete story, and include success metrics.

6. User interaction and design

After the team fleshes out the solution for each user story, link design explorations and wireframes to the page.

7. Questions

As the team understands the problems to solve, they often have questions. Create a table of "things we need to decide or research" to track these items.

8. What we're not doing

Keep the team focused on the work at hand by clearly calling out what you're not doing. Flag things that are out of scope at the moment, but might be considered at a later time.



PRO TIP

The Agile Manifesto encourages flexibility in creating requirements. Teams might use user story mapping or collaborate directly with customers to identify problems and brainstorm solutions.

Regardless of the approach, requirements are just one tool for defining and communicating customer needs. For more, see our section on Agile design and how product owners can use Keynote or PowerPoint to mock up requirements.

Benefits of a well written product requirements document

If you are to take away anything from this blog, understand the “why” – the not “what” – because the “why” will help you explore what is best for your team.

Here are the benefits and challenges we’ve observed with the one-page dashboard approach:

1. One page, one source

Keeping it simple. The product requirements document becomes the “landing page” for everything related to the set of problems within a particular epic.

Having something that is the central go-to location saves your team members time in accessing this information and gives them a concise view.

2. Extra agility

One of the awesome things about using a simple page to collaborate (verses a dedicated requirements management tool) is that you can be agile about your [documentation](#)! You don’t have to follow the same format every time – do what you need, when you need it, and be agile about it. Chop and change as needed.

3. Just enough context and detail

We often forget how powerful a simple link can be. We embed a lot of links within our product requirements documents. It helps abstract out the complexity and progressively disclose the information to the reader as needed.

Linking detailed resources may include such things as:

- Customer interviews for background, validation or further context for the feature
- Pages or blogs where similar ideas were proposed

- Previous discussion or technical documentation and diagrams
- Videos of product demos or other related content from external sources

4. Living stories

Once the stories have been roughly thought out and entered as work items in Jira, we link to them in our page (which, conveniently, creates a link from the work items back to the page as well).

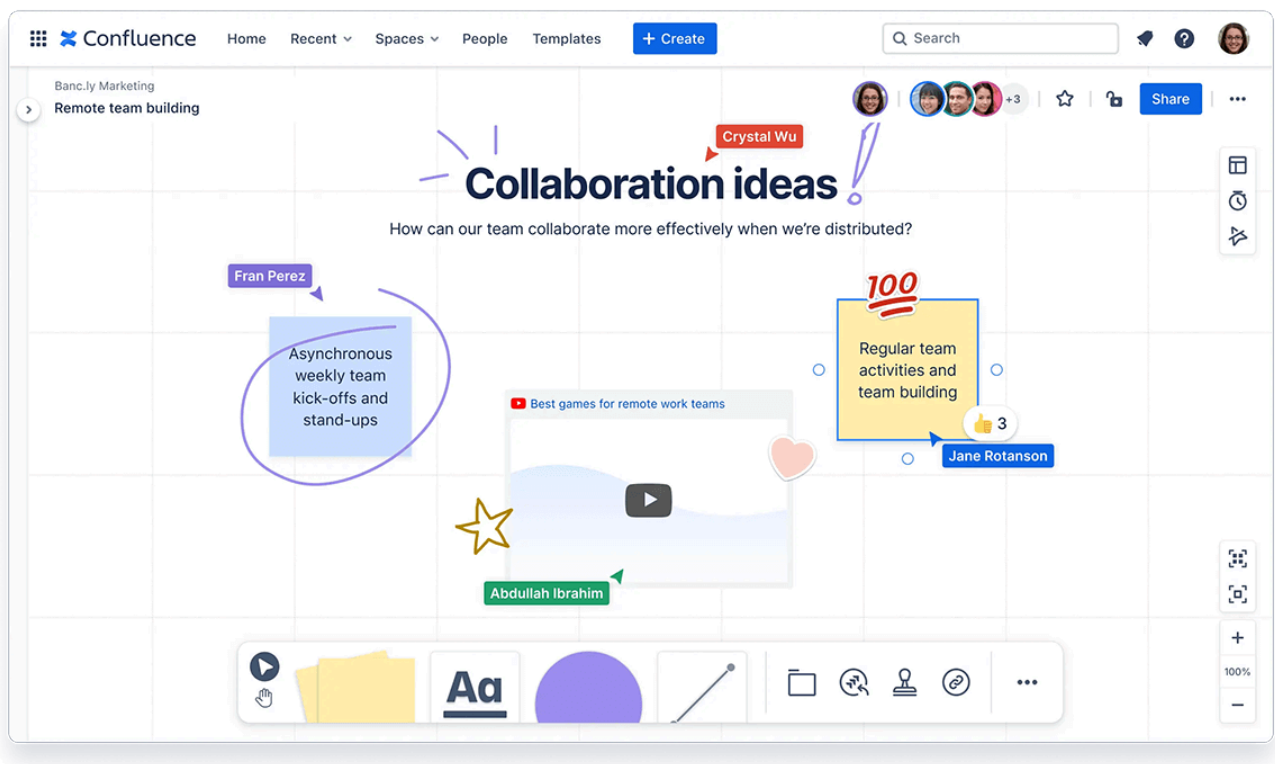
The two-way syncing between [Confluence](#) and [Jira](#) means we automatically get to see each work item's current status right from the requirements page.

5. Collective wisdom

Capturing product requirements in Confluence makes it easy for other people in different teams to contribute and make suggestions. I've been amazed at the number of times someone from another team jumped into the conversation with a comment providing great feedback, suggestions, or lessons learned from similar projects.

It helps a large organization feel like a small team.

6. Engaging 'extras'



Diagrams made with tools like [Confluence Whiteboards](#) better communicate the problems to your team. You can also embed external images, videos, and dynamic content.

7. Collaboration

The most important aspect of all this is getting everyone involved. Never write a product requirements document by yourself—you should always have a developer with you and write it together. Share the page with your team and get feedback.

Comment, ask questions, encourage others to contribute with thoughts and ideas. This is especially important for distributed teams who don't often get a chance to discuss projects in person.

The challenges product requirements documents

With every approach there are down-sides. Here there are two main challenges we've experienced and observed from customers as well:

1. Documentation can go stale

What happens when you implement a story and get feedback and then modify the solution? Does someone go back and update the requirements page with the final implementation?

This is a challenge with any type of documentation, and it's always worth questioning whether such trade-offs are worthwhile. Talk to your team about what you would do in a scenario like this.

2. Lack of participation

“What can I do to encourage people to comment?” “How can I encourage people to write more specs and stories on our intranet?”

This is a tough nut to crack, and it comes down to various wiki adoption techniques in your organization. There are plenty of resources to help you here. There may be deeper cultural issues at play here, too.

Get to work on your PRD with Jira and Confluence

When requirements are nimble, the product owner has more time to understand and keep pace with the market. And keeping them informative-but-brief empowers the

development team to use whatever implementation fits their architecture and technology stack best.

Once a project's requirements are reasonably well-baked, we recommend linking the user stories to the corresponding stories in the development team's work tracker.

T

Ready-made Jira templates

Browse our library of custom Jira templates for various teams, departments, and workflows.

[Go to Jira templates →](#)

A comprehensive introduction to Jira

Use this step-by-step guide to discover essential features and the best practices to maximize your productivity.

[Go to Jira Product guide →](#)

Remember, be Agile in your evolution of requirements for a project. It's okay to change user stories as the team builds, ships, and gets feedback. Always maintain a high

Understanding the Basics of Git

From beginners to advanced experts, use this guide to Git to learn the basics with helpful tutorials and tips.

[Learn more →](#)



Company

Careers

Events

Blogs

Investor Relations